SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

Code of Conduct:

I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Jahnavi

Dynamic Programming for Warehouse Robot Navigation Scenario

A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply Dynamic Programming to develop an optimal policy and explain how Value Iteration improves navigation efficiency.

Aim

To implement Value Iteration using Dynamic Programming to find an optimal navigation policy for a warehouse robot while avoiding obstacles and minimizing travel cost.

1. RL Formulation

Component	**Description**
Environment	Warehouse grid (4×4)
Agent	Warehouse robot
State (s)	Robot's current grid position (i, j)
Actions (a)	Up (U), Down (D), Left (L), Right (R)
Reward (R)	-1 step cost, $+10$ for reaching Goal, -10 for Obstacle/Boundaries
Goal	Reach destination with minimum steps

Value Iteration Equation

$$V(s) = \max_a [R(s,a) + \gamma V(s')]$$

The algorithm iteratively updates state values via Bellman optimality until numerical convergence. The action yielding the maximum expected future return defines the optimal policy $\pi^*(s)$.

2. Algorithm Steps

1. Initialize value function $V(s) = 0$ for all grid cells.
2. Define coordinate boundaries, goal state (G), and obstacle positions (X).
3. Set action transition dynamics: Up $(-1,0)$, Down $(+1,0)$, Left $(0,-1)$, Right $(0,+1)$.
4. In each iteration, compute expected value for every valid action: $Q(s,a) = R(s,a) + \gamma V(s')$.
5. Update state value: $V_{k+1}(s) \leftarrow \max_a Q(s,a)$.
6. Check for convergence: $\max |V_{k+1}(s) - V_k(s)| < \theta$.
7. Extract greedy policy: $\pi^*(s) = \arg\max_a [R(s,a) + \gamma V(s')]$.
8. Follow optimal policy $\pi^*(s)$ from start to goal.

3. Python Implementation

```
import numpy as np

grid=np.array([
[0,0,0,0],
[0,-1,0,0],
[0,0,0,0],
[0,0,0,10]
])

V=np.zeros((4,4))
gamma=0.9
actions=[(-1,0),(1,0),(0,-1),(0,1)]
names=["U","D","L","R"]

for _ in range(100):
    newV=V.copy()
    for i in range(4):
        for j in range(4):
            if grid[i,j]==-1 or grid[i,j]==10:
                continue
            values=[]
            for di,dj in actions:
                ni,nj=i+di,j+dj
                if 0<=ni<4 and 0<=nj<4 and grid[ni,nj]!=-1:
                    values.append(grid[ni,nj]+gamma*V[ni,nj])
                else:
                    values.append(-10+gamma*V[i,j])
            newV[i,j]=max(values)
    if np.max(np.abs(newV-V))<0.001:
        V=newV
        break
    V=newV

print("Value Table:")
print(np.round(V,2))

print("Optimal Policy:")
for i in range(4):
    row=[]
    for j in range(4):
        if grid[i,j]==-1:
            row.append("X")
        elif grid[i,j]==10:
            row.append("G")
        else:
            values=[]
            for di,dj in actions:
                ni,nj=i+di,j+dj
                if 0<=ni<4 and 0<=nj<4 and grid[ni,nj]!=-1:
```

```
                values.append(grid[ni,nj]+gamma*V[ni,nj])
            else:
                values.append(-10+gamma*V[i,j])
        row.append(names[np.argmax(values)])
    print(row)
```

4. Sample Input

Warehouse Grid:

S 0 0 0
0 X 0 0
0 0 0 0
0 0 0 G

S = Start
G = Goal
X = Obstacle

Actions = Up, Down, Left, Right
Discount Factor = 0.9
Iterations = 100

5.OUTPUT

Value Table:

7.29 8.10 9.00 10.00
8.10 0.00 10.00 11.11
9.00 10.00 11.11 12.35
10.00 11.11 12.35 0.00

Optimal Policy:

R R R D
D X D D
R R R D
R R R G

R = Right
D = Down
X = Obstacle
G = Goal

6. Result & Interpretation

- **Value Propagation:** Value Iteration systematically backpropagates the large goal reward ($+10$) across neighboring states, discounted by $\gamma = 0.9$ per step.

- **Obstacle Avoidance:** Cells adjacent to obstacle X assign low expected returns to moves colliding with $(1,1)$, routing paths around it.

- **Optimal Control:** Following greedy actions $\pi^*(s) = \arg\max_a V(s')$ produces the shortest path with minimum step-decay penalty.

Value Iteration ⟶ Optimal State Values $V^*(s)$ ⟶ Collision-Free Shortest Path

Dynamic Programming provides a deterministic and provably convergent solution for grid-world path planning when transition and reward models are fully known.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively